

Informe de Laboratorio N° 02

Sistema de Control Proporcional de Nivel de Agua

Sistemas de Control Automático

Ingeniería en Informática

Universidad Nacional de Avellaneda

Basso Joel, Candelieri Santiago, Raab Germán

El informe desarrolla adecuadamente lo realizado en la práctica. Se logró controlar adecuadamente el nivel de agua y reducir los problemas del sensor y la transferencia de la bomba.

Una corrección importante fue precisar cómo la constante de la bomba está incorporada a la constante de proporcionalidad del lazo de control. Esta constante de la bomba K_B no había sido incluida dentro de las simulaciones, lo que explica la diferencia que se ve en el gráfico respecto de los valores medidos en cuanto al error estacionario.

Otra cuestión importante es el filtro implementado en el código para reducir el ruido del sensor. Este código fue elaborado a partir de las mediciones del TL1. En las figuras 7 a 10 se muestra que este filtro parte siempre de una distancia inicial igual a cero; cuestión que podría mejorarse.

Las conclusiones son adecuadas.

Ing. G. Caporaletti - J. Castellano
APROBADO (2C 2018)

Índice

1 Introducción	2
2 Desarrollo	2
2.1 Introducción Teórica	2
2.1.1 Sistema Hídrico	2
2.1.2 Realimentación Proporcional	4
2.2 Funcionamiento del sistema	5
2.3 Funcionamiento del Control	6
2.4 Medición caudal máximo	7
2.5 Simulaciones	10
3 Conclusión	11
Anexo I - Código de Arduino	12

1 Introducción

El siguiente informe tiene como objetivo la implementación de un sistema de control con realimentación proporcional sobre el ya estudiado sistema de nivel de agua, monitoreado a través de un sistema embebido utilizando Arduino UNO.

Se reemplazará su control on/off estudiado previamente por uno de tipo proporcional.

2 Desarrollo

2.1 Introducción Teórica

2.1.1 Sistema Hídrico

Se considera un tanque de agua, llenado a través de una bomba con cierto caudal Q_i . Dicho tanque es desagotado por medio de una tubería que posee una válvula, con una resistencia R_h al paso del agua.

Tanto la altura h (distancia entre el nivel de agua en el tanque y la válvula) como la resistencia hídrica de la tubería y la válvula, definen el caudal que saldrá del tanque (Q_o).

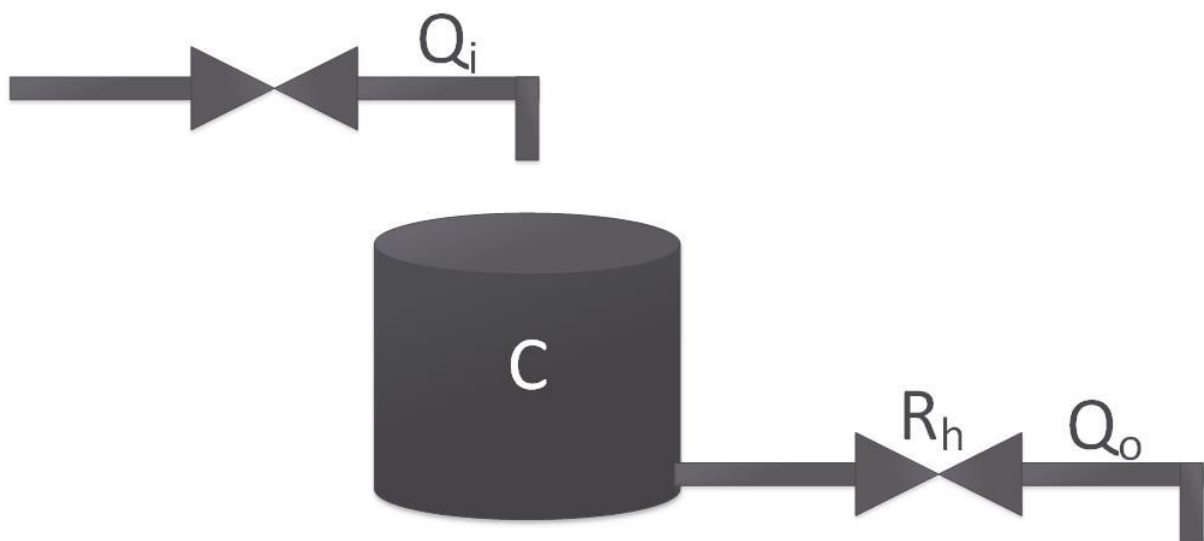


Figura 1. Diagrama del sistema.

Haciendo una analogía con circuitos eléctricos, dado que la altura a la que se encuentra el nivel de agua respecto de la válvula genera un potencial, y que el caudal de salida del tanque es limitado por la resistencia hídrica de la válvula, se podría utilizar la Ley de Ohm para hallar el caudal de salida:

$$Q_o = \frac{h}{R_h}$$

Por lo tanto, dado que el caudal tiene como unidad m^3/s y la altura m , la resistencia hídrica de la válvula tendrá como unidad s/m^2 .

Por otro lado, se sabe que el nivel de agua dependerá de la diferencia entre el caudal de entrada y el de salida. También el tanque, al comenzar vacío, tardará en llenarse, provocando que el caudal de salida en un principio sea débil y se vaya estabilizando con el tiempo. Entonces, continuando con la analogía del circuito eléctrico, se podría considerar al tanque como un capacitor, quedando definida la altura como:

$$h = \frac{1}{C} \int (Q_i - Q_o) dt$$

Donde C es la capacitancia del tanque y se mide en m^2 .

De esta manera, reemplazando h en la primer fórmula y aplicando la Transformada de Laplace, se busca llegar a una relación entre el caudal de salida y el de entrada (la función de transferencia del sistema):

$$Q_o(t) = \frac{1}{R_h C} \int (Q_i(t) - Q_o(t)) dt$$

$$L\{Q_o(t)\} = \frac{1}{R_h C} L\left\{\int (Q_i(t) - Q_o(t)) dt\right\}$$

$$Q_o(s) = \frac{1}{R_h C s} ((Q_i(s) - Q_o(s)))$$

$$\frac{Q_o(s)}{Q_i(s)} = \frac{1}{R_h C s + 1}$$

$$\Rightarrow G(s) = \frac{H(s)}{Q_i(s)} = \frac{Q_o(s) \cdot R_h}{Q_i(s)} = \frac{R_h}{R_h C s + 1}$$

De esta manera, el diagrama de bloques del sistema es:

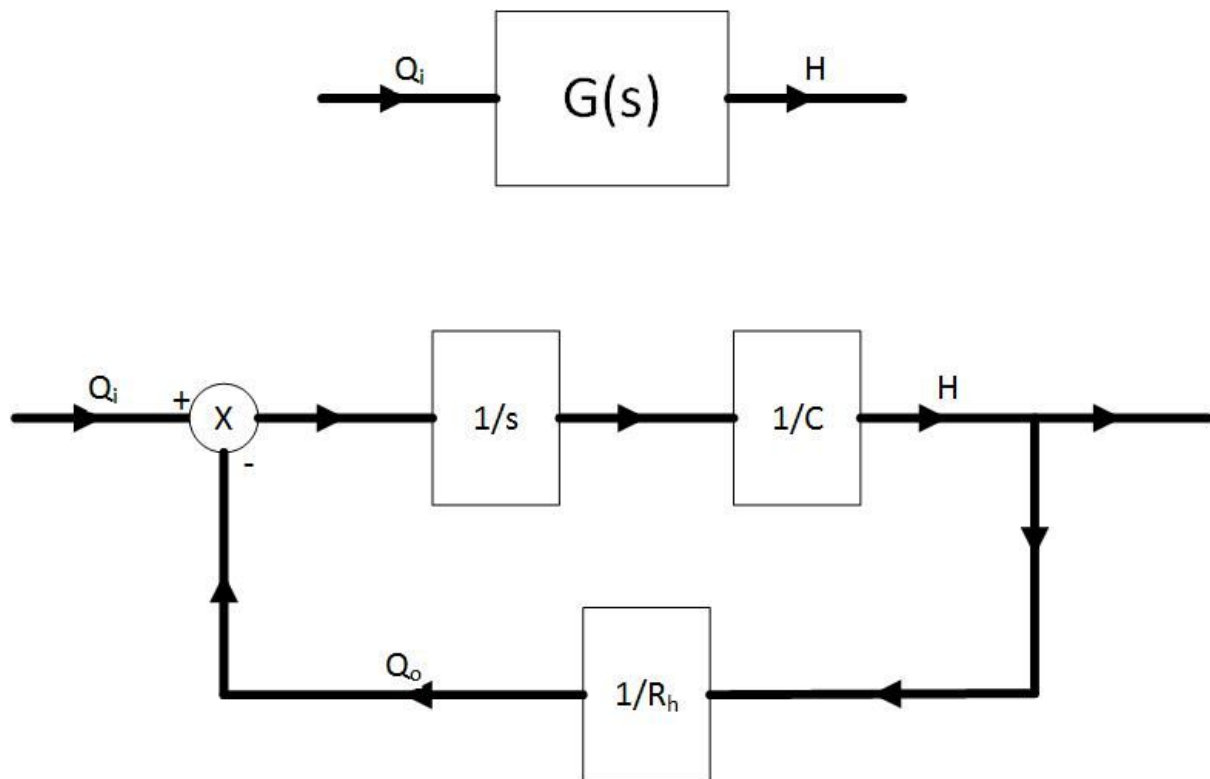


Figura 2. Diagrama de bloques del sistema de nivel de agua.

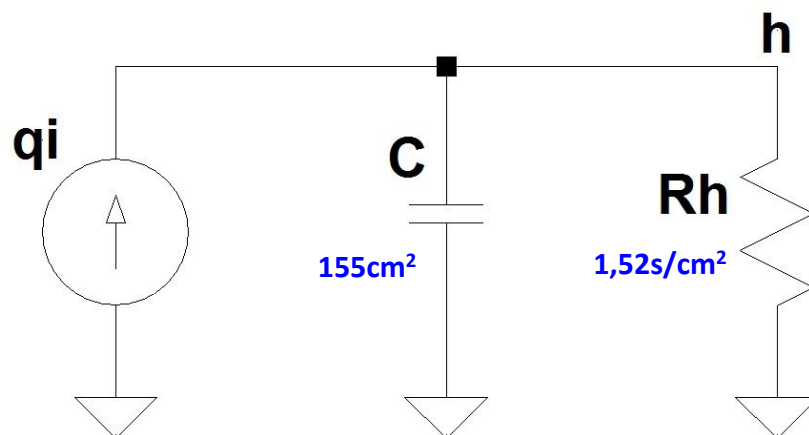


Figura 3. Diagrama del circuito eléctrico equivalente al sistema.

2.1.2 Realimentación Proporcional

Considerando el sistema de la **Figura 1**, se necesitará tomar como entrada una referencia de altura deseada en lugar de un caudal (para hacer al sistema más sencillo de controlar). De esta manera, el controlador se encargará de regular el caudal necesario para llegar a la altura ingresada.

El sistema deberá analizar la diferencia entre la altura de salida y la deseada, para ejecutar una acción de control proporcional a esta diferencia: por ejemplo, si la diferencia de altura es 0, el sistema no entregará caudal de agua, porque ya llegó a la altura. En cambio, si la diferencia es muy grande, aumentará el caudal de agua en un valor alto (idealmente).

Por lo tanto, el diagrama de bloques del nuevo sistema es el siguiente:

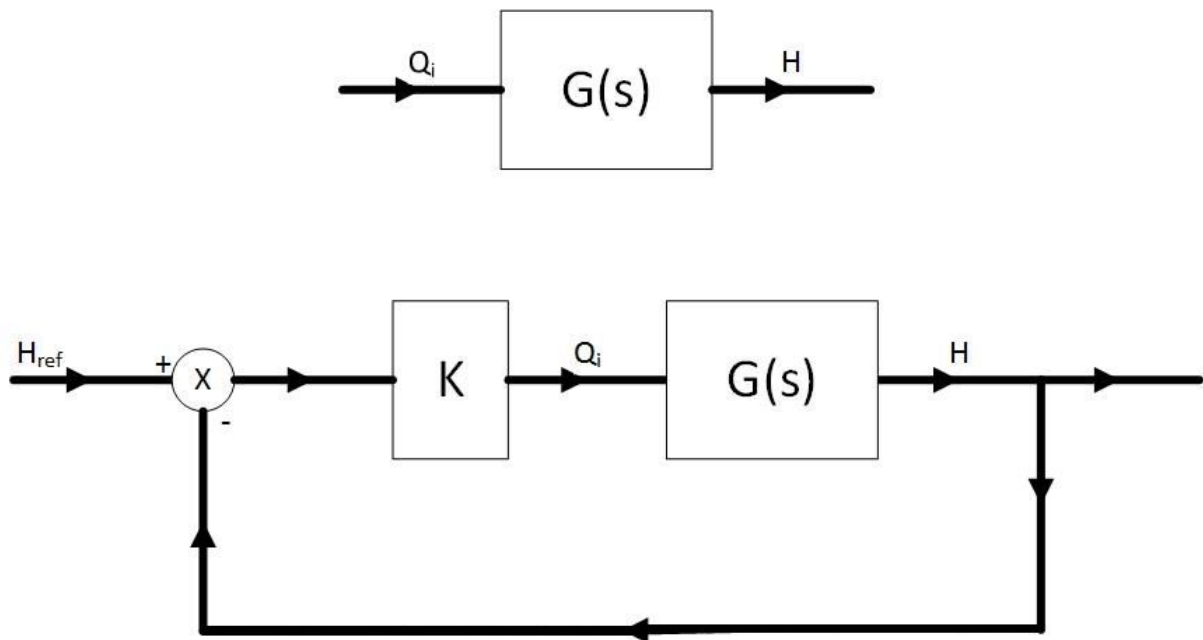


Figura 4. Diagrama del sistema de nivel de agua con realimentación proporcional.

Y la función de transferencia de este nuevo sistema, considerando que posee de entrada una altura de referencia y como salida la altura propiamente dicha: **(1)**

$$M(s) = \frac{K \cdot G(s)}{1 + K \cdot G(s)}$$

$$M(s) = \frac{K \cdot R_h}{K \cdot R_h + 1 + R_h \cdot C \cdot s}$$

2.2 Funcionamiento del sistema

El sistema está compuesto por:

- Dos recipientes (uno dentro de otro).
- Una bomba de agua de 5V.
- Mangueras.
- Un sensor de ultrasonido HC-SR04.
- Un transistor.

(1) La constante K señalada en la figura 4 y en la ecuación de M(s) incluye no sólo una constante de proporcionalidad implementada dentro del código del Arduino, sino también la constante de proporcionalidad propia de la bomba de agua.

- Un microcontrolador Arduino UNO.
- Una fuente de 5V (para alimentar la bomba).

La bomba se encarga de llevar el agua hacia el recipiente que se encuentra más elevado. Este recipiente tiene un orificio con una manguera por la cual se vacía el contenido del mismo completando el circuito. El sensor de ultrasonido se encarga de medir la altura del agua dentro del recipiente y enviarla al arduino. Con el dato de la altura, a través de la lógica del código desarrollado, se decide si se debe activar o desactivar la bomba de agua, y se envía la señal correspondiente. El transistor es el encargado de activar la bomba de agua según la señal proveniente del arduino.

2.3 Funcionamiento del Control

El control se hace por medio de un microcontrolador arduino acompañado por un sensor de ultrasonido HC-SR04 y un transistor TIP41. El arduino contiene el programa que se encarga de tomar periódicamente la distancia que existe entre el sensor y el líquido. Esta altura se compara con la altura del líquido deseada: Dicho valor se multiplica por una constante (establecida previamente a través de experimentación) que indica con cuánta potencia tiene que funcionar la bomba (utilizando PWM para controlarla). El código utilizado se puede ver en el Anexo I.

2.4 Medición caudal máximo

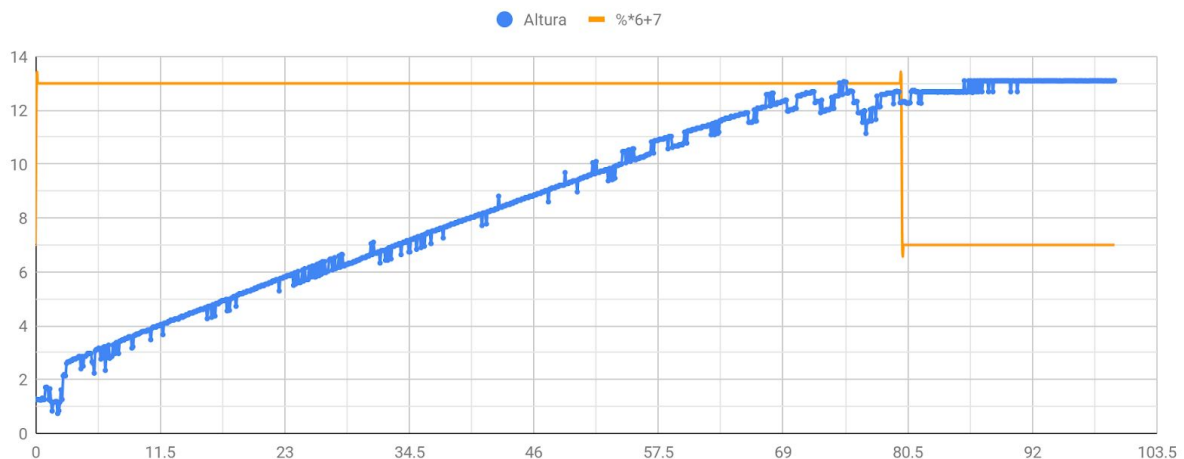


Figura 5. Llenado completo del tanque con la bomba a potencia máxima.

Se tapo el orificio inferior del recipiente y se activó la bomba a máxima potencia. Concluimos que el caudal máximo de la bomba es aproximadamente $22\text{cm}^3/\text{s}$ ya que el nivel se elevó 6cm en aproximadamente 41s y el área del recipiente es de 155cm^2 . (2)

Debido a que la relación entre la tensión que se le entrega a la bomba y el caudal que produce la misma no es lineal, nos vimos obligados a “linealizarlo” realizando un reescalado del porcentaje de tensión con que se activa la misma.

Luego de varias pruebas notamos que por debajo del 25% la bomba no encendía, ya que la corriente no era suficiente como para realizar la purga de la manguera, que llega al recipiente, ni para mantener un nivel de líquido estable dentro de la misma. Por esto es que a continuación clasificamos los gráficos de las mediciones con el título “Reescalado”, indicando que el código con el que se realizó el control tiene en cuenta esto.

(2) La primer medición graficada en la figura 5 tiene la importancia de incorporar en nuestro análisis la constante de proporcionalidad que introduce la bomba, el cual resultó: $K_B=22\text{cm}^3/\text{s}$ (cuando tiene una entrada de 1 o 100%). Esta constante por la constante programada en el Arduino me da la constante K del control proporcional.

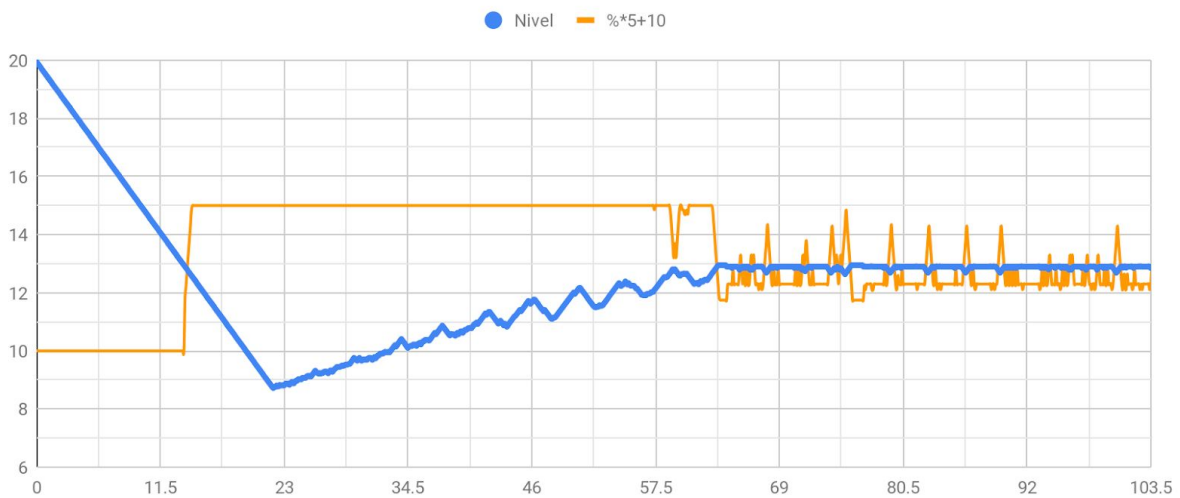


Figura 7. Funcionamiento del sistema reescalado, con $k = 2$ y $ref = 13$.
En realidad este gráfico corresponde a: $K=6 \cdot 22 \text{ cm}^2/\text{s}$

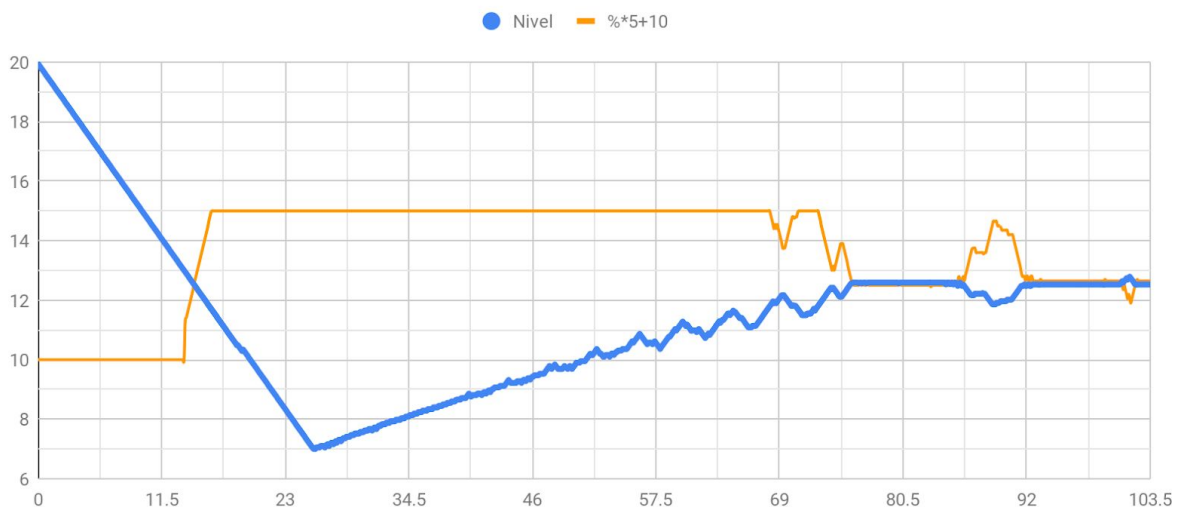


Figura 8. Funcionamiento del sistema reescalado, con $k = 6$ y $ref = 13$.
En realidad este gráfico corresponde a: $K=2 \cdot 22 \text{ cm}^2/\text{s}$

Para comparar el error de estado estacionario con estos gráficos tenemos que evaluar cuánto debiera dar según la transferencia $M(s)$. Para t tendiendo a infinito:
 $h(\text{en estado estacionario}) = h_{\text{REF}}(\text{final}) \cdot M(0)$. A su vez: $M(0) = K \cdot R_H / (1 + K \cdot R_H)$.
 Para $K = 2 \cdot 22 \text{ cm}^2/\text{s}$, sería: $M(0) = 0,9853$ (un error del 1,5%). Y con $h_{\text{REF}}(\text{final}) = 13 \text{ cm}$, sería:
 $h(\text{en estado estacionario}) = 12,8 \text{ cm}$. Lo cual coincide con la figura 8 aproximadamente.
 Para $K = 6 \cdot 22 \text{ cm}^2/\text{s}$, sería: $M(0) = 0,9950$. Y con $h_{\text{REF}}(\text{final}) = 13 \text{ cm}$, sería: $h(\text{en estado estacionario}) = 12,9 \text{ cm}$. Lo cual también coincide con la figura 7 aproximadamente.

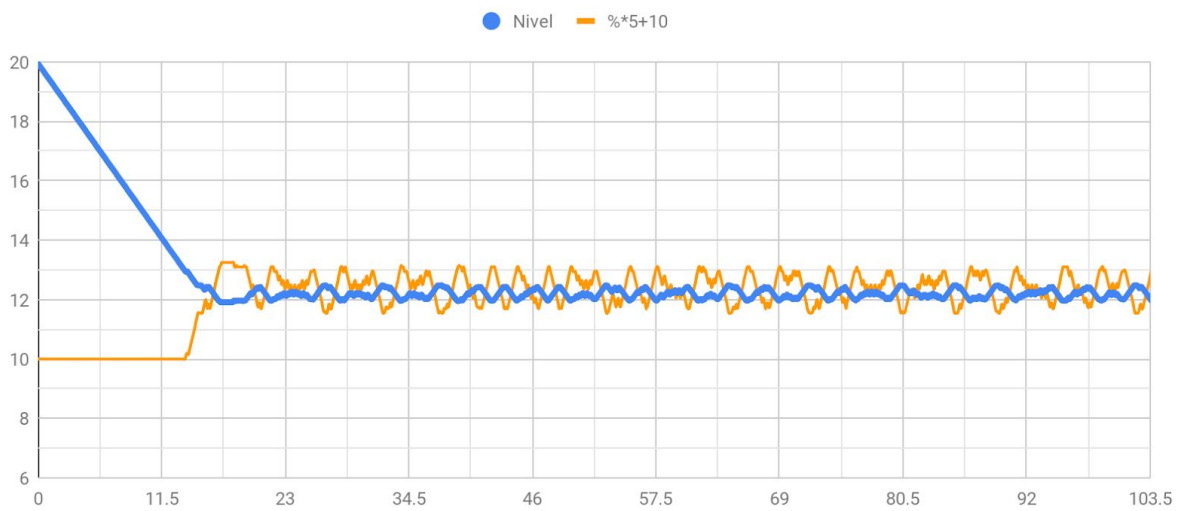


Figura 9. Funcionamiento del sistema, con $k = 6$ y $ref = 13$.

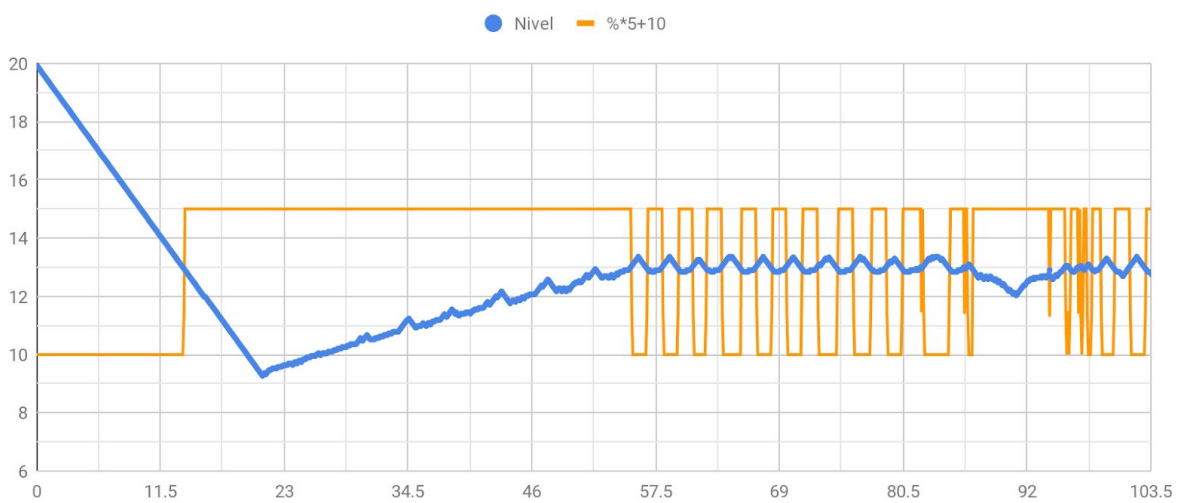


Figura 10. Funcionamiento del sistema reescalado, con $k = 20$ y $ref = 13$.

2.5 Simulaciones

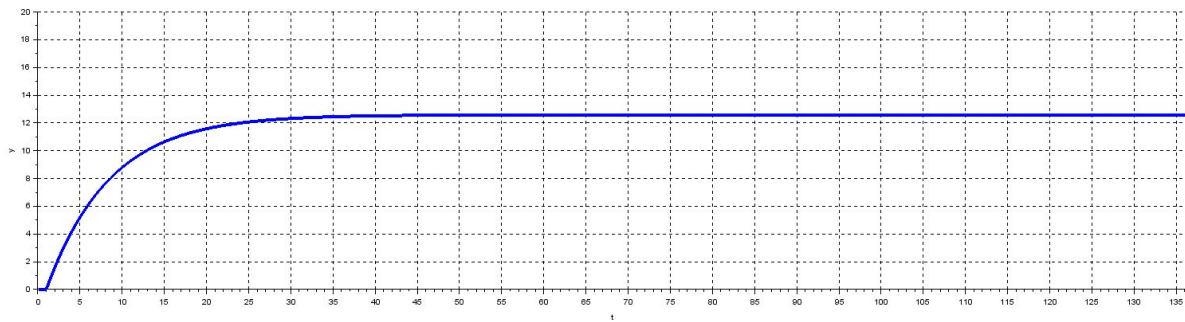


Figura 11. Simulación de la respuesta, con $k = 20$ y $ref = 13$.

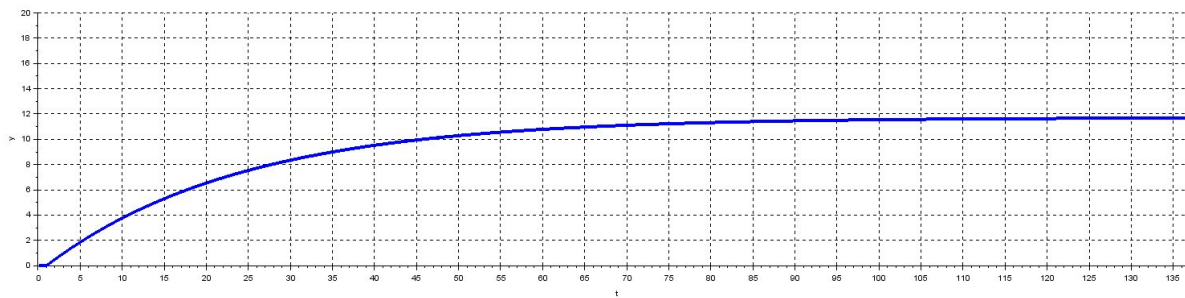


Figura 11. Simulación de la respuesta, con $k = 6$ y $ref = 13$.

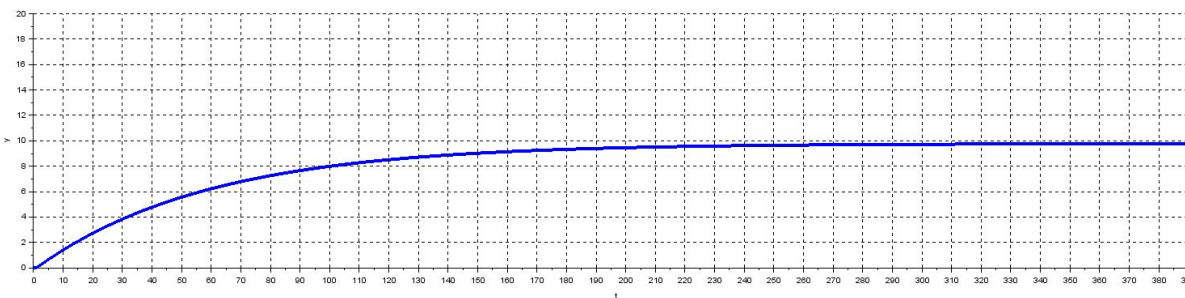


Figura 11. Simulación de la respuesta, con $k = 2$ y $ref = 13$.

3 Conclusión

Se logró realizar las modificaciones al sistema (con respecto a la versión simplificada anterior). Como resultado, logramos una implementación con control proporcional, que nos permitió mantener un nivel de agua preestablecido, con la bomba encendida todo el tiempo (pero variando su potencia), a diferencia del control ON-OFF que nos obligaba a operar en un ciclo de histéresis.

Por otro lado, variando la constante de proporcionalidad, observamos las diferentes respuestas, en las cuales se ve que la señal de control es menor cuando se está llegando al valor de referencia en el nivel de agua. Así como también observamos que para un K muy grande, el sistema funciona como un ON-OFF.

A la conclusión podría agregarse:

Simulación

Filtro

Anexo I - Código de Arduino

```
#include <HCSR04.h>
#include "RangeFilter.h"

RangeFilter rangeFilter(23.0f, 2.0f, 0.5f);
HCSR04 hc(2, 3); // Initialization class HCSR04(trig pin, echo pin)
unsigned long currentTime = 0;
float currentDist = 10.0f;
int pinBomba = 5;
float referencia = 7;
float estado = 0.0f;
float k = 20.0f;
float error = 0.0f;

void printMonitor()
{
    Serial.print(rangeFilter.GetCurrentValue());
    Serial.print("\t");
    Serial.print(estado);
    Serial.print("\t");
    Serial.print(error);
    Serial.println("");
}

void setup()
{
    Serial.begin(9600);
    pinMode(pinBomba, OUTPUT);
}

void loop()
{
    delay(100);
    currentTime = millis();
    currentDist = hc.dist();
    rangeFilter.SetCurrentValue(currentTime, currentDist);
    printMonitor();
    error = rangeFilter.GetCurrentValue() - referencia;
    estado = error * k;
    estado = min(estado, 0.75f);
    estado = max(estado, 0.0f);
    if(estado > 0.0f)
        estado += 0.25f;
    analogWrite(pinBomba, estado*255);
}
```

```
#ifndef RANGEFILTER_h
#define RANGEFILTER_h

#include "Arduino.h"

class RangeFilter
{
private:
    unsigned long time;
    float value;
    float maxValue;
    float minValue;
    float maxRate;

public:
    RangeFilter(float maxValue, float minValue, float maxRate);
    void SetCurrentValue(unsigned long newTime, float newValue);
    float GetCurrentValue();
};

#endif
```

```
#include "Arduino.h"
#include "RangeFilter.h"

RangeFilter::RangeFilter(float maxValue, float minValue, float maxRate)
{
    this->maxValue = maxValue;
    this->minValue = minValue;
    this->maxRate = maxRate;
}

void RangeFilter::SetCurrentValue(unsigned long newTime, float newValue)
{
    if(newValue <= maxValue && newValue >= minValue)
    {
        float valueDiff = this->maxRate * (float)(newTime - this->time) / 1000;
        float maxValueDiff = this->value + valueDiff;
        float minValueDiff = this->value - valueDiff;
        this->value = min(newValue, maxValueDiff);
        this->value = max(this->value, minValueDiff);
    }
    this->time = newTime;
}

float RangeFilter::GetCurrentValue()
{
    return this->value;
}
```